

ParkVault

Smart Parking Management System

MCA Final Year Project
Complete Technical Documentation

Technology: Python · Flask · SQLite · SQLAlchemy · Jinja2 · HTML5 / CSS3

Domain	Parking Management / Smart City
Platform	Web Application (Browser-based)
Backend	Python 3 + Flask Framework
Database	SQLite via SQLAlchemy ORM
Frontend	HTML5, CSS3, Vanilla JavaScript
Users	Regular Users + Admin Panel

Table of Contents

- 1. Project Overview**
- 2. Technology Stack & Dependencies**
- 3. Project File Structure**
- 4. Database Design — The Four Models**
- 5. Application Architecture — MVC Pattern**
- 6. Control Flow — Operation by Operation**
 - 6.1 Application Startup
 - 6.2 User Registration
 - 6.3 Login Flow
 - 6.4 Search for Parking
 - 6.5 Spot Selection
 - 6.6 Reservation (Booking)
 - 6.7 Check-Out / Billing
 - 6.8 Admin — Force Release
 - 6.9 Admin — Lot Management
 - 6.10 Reports & Analytics
 - 6.11 CSV Export
- 7. Authentication & Security Layers**
- 8. Theme System (Dark / Light Mode)**
- 9. Data Flow Diagram**
- 10. Key Design Decisions**

1. Project Overview

ParkVault is a web-based **Smart Parking Management System** designed for urban parking lots. It serves two types of users — **regular users** (drivers) who find and reserve parking spots, and **administrators** who manage lots, users, and revenue reports. The system tracks reservations in real time and automatically calculates billing per minute based on actual parking duration using server-side timestamps.

The application is built entirely with open-source technologies, requiring no paid services, no cloud infrastructure, and no frontend build tools — making it ideal for academic deployment and demonstration on any machine with Python installed.

2. Technology Stack & Dependencies

Layer	Technology	Purpose
Backend	Python 3.14 + Flask	Web server, routing, all business logic
Database	SQLite via SQLAlchemy ORM	Persistent data storage — single file, zero config
Authentication	Flask-Login + Flask-Bcrypt	Session management, bcrypt password hashing
Templating	Jinja2 (built into Flask)	Dynamic HTML rendered on the server side
Frontend	HTML5 + CSS3 + Vanilla JS	UI and interactivity (no JS framework needed)
Styling	Custom CSS (Glassmorphism)	Dark/light theme, responsive layout
Package Manager	pip + virtualenv (.venv)	Isolated Python dependency environment

Python packages (requirements.txt):

- **flask** — The core web framework — handles routing, request/response, templating
- **flask-sqlalchemy** — ORM bridge — maps Python classes to SQLite database tables
- **flask-login** — Manages logged-in user sessions using secure HTTP cookies
- **flask-bcrypt** — Hashes passwords using the bcrypt algorithm before storing in DB

3. Project File Structure

Paking_app/

■

■■■ run.py

■■■ models.py

← Main application (all routes + business logic)

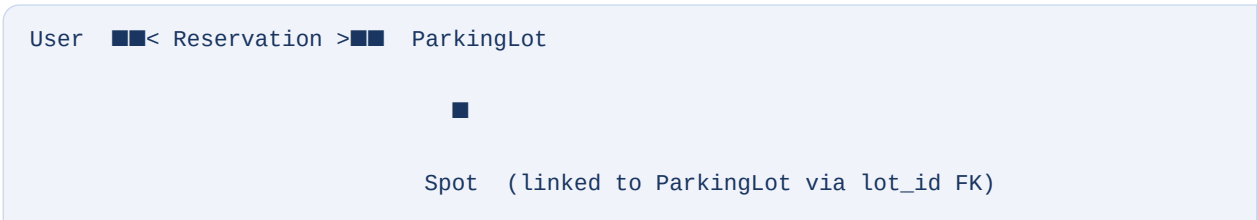
← Database schema (4 tables as Python classes)

■■■ style.css	← Complete stylesheet (dark + light theme)
■	
■■■ index.html	← Login page
■■■ register.html	← User registration page
■■■ user-dashboard.html	← User home after login
■■■ search-parking.html	← Find available parking lots
■■■ spot-selection.html	← Visual spot grid for a chosen lot
■■■ reservation-history.html	← User's booking history (table + timeline)
■	
■■■ admin-dashboard.html	← Admin home (live reservations overview)
■■■ admin-lots.html	← Manage parking lots (add/edit/delete)
■■■ admin-users.html	← Manage registered users
■■■ admin-reports.html	← Revenue analytics + top users
■■■ admin-database.html	← Raw DB browser + live SQL workspace
■	
■■■ instance/	
■■■ parkvault.db	← SQLite database file (auto-created on startup)

*Note: Flask is configured with **template_folder='.'** and **static_folder='.'**, so all HTML and CSS files sit in the project root rather than separate templates/ and static/ directories.*

4. Database Design — The Four Models

All tables are defined in **models.py** using Flask-SQLAlchemy as Python classes. SQLAlchemy translates these into SQL CREATE TABLE statements automatically. The four models and their relationships are:



4.1 User Table

Column	Type	Description
id	Integer PK	Auto-incrementing primary key
first_name, last_name	String(60)	User's full name — combined as user.name property
email	String(120) UNIQUE	Used as the login credential — must be unique
password_hash	String(256)	bcrypt hash of the password — never stored as plain text
phone	String(20)	Optional contact number
vehicle	String(50)	Vehicle registration number shown in reservations
status	String	'active' / 'parked' / 'suspended' — reflects current state
is_admin	Boolean	True → Admin Portal access; False → User Portal
joined	DateTime	Auto-set to current timestamp at account creation

4.2 ParkingLot Table

Column	Type	Description
id	Integer PK	Auto-incrementing primary key
name	String(100)	Lot display name, e.g. 'Sarojini Nagar Hub'
address, pin_code	String	Location info — used in search/filter
lot_type	String	'open' / 'covered' / 'multi-level'
total_spots	Integer	Used to calculate occupancy percentage
rate_per_hour	Float	e.g. 30.0 (₹) — used in billing formula
status	String	'active' (shown in search) / 'inactive' (hidden)

4.3 Spot Table

Column	Type	Description
id	Integer PK	Auto-incrementing primary key
lot_id	FK → parking_lot.id	Which lot this spot belongs to
spot_number	String(10)	e.g. 'A-01', 'B-07' — Row-Number format
is_occupied	Boolean	True when a car is currently parked; False when free

A UNIQUE constraint on (lot_id, spot_number) prevents duplicate spot records per lot.

4.4 Reservation Table

Column	Type	Description
id	Integer PK	Auto-incrementing primary key
user_id	FK → user.id	Which user made this reservation
lot_id	FK → parking_lot.id	Which lot they parked in
spot_number	String(10)	Which specific spot (e.g. 'C-04')
check_in	DateTime	Server time when reservation was created
check_out	DateTime (nullable)	Server time when user released the spot
cost	Float (nullable)	Calculated only at check-out; NULL while active
status	String	'active' / 'completed' / 'cancelled'

5. Application Architecture — MVC Pattern

ParkVault follows the **Model-View-Controller (MVC)** architectural pattern:

- **Model** — The four SQLAlchemy classes in models.py (User, ParkingLot, Spot, Reservation)
- **View** — The HTML template files, rendered dynamically by Jinja2
- **Controller** — The route functions in run.py, which receive requests, query the DB, and return rendered pages

Browser (View)

- HTTP GET / POST request



Flask Route in run.py (Controller)

- Query / Update via SQLAlchemy



SQLite Database (Model)

- Returns data objects



Jinja2 renders HTML template (View)

- HTTP Response with complete HTML page



Browser displays the page

There is **no separate REST API**. Data is embedded directly into the HTML by Jinja2 using template tags like `{{ variable }}`, `{% for %}`, and `{% if %}`. JavaScript is only used for client-side UI interactions (spot grid rendering, modals, theme toggle) — not for fetching data.

6. Control Flow — Operation by Operation

6.1 Application Startup

```
$ python run.py
```

→ Flask app object created with SECRET_KEY and DB URI

→ `db.create_all()` ← creates all 4 tables if they don't exist

→ `seed_db()` ← checks if DB is empty; if so, inserts:

2 admin accounts + 8 regular users

6 parking lots (5 active, 1 inactive)

310 spot records auto-generated using naming formula:

Spots 1-10 → Row A (A-01 to A-10)

Spots 11-20 → Row B (B-01 to B-10) ... and so on

→ Flask dev server starts on port 5001

6.2 User Registration Flow

```
GET /register
```

→ render `register.html` (blank form shown)

```
POST /register (form submitted)
```

→ Validate: do passwords match?

→ Check: does email already exist in User table?

→ `bcrypt.generate_password_hash(password)` ← one-way hash with salt

→ New User object saved to DB

```
→ login_user(user) ← Flask-Login sets session cookie in browser

→ redirect to /dashboard
```

6.3 Login Flow

```
GET /

→ If already logged in → redirect to /admin or /dashboard

→ Else → render index.html (login form)

POST /login

→ User.query.filter_by(email=email).first() ← DB lookup

→ bcrypt.check_password_hash(stored_hash, entered_password)

→ If match → login_user(user)

    is_admin=True → redirect to /admin

    is_admin=False → redirect to /dashboard

→ If no match → flash('Invalid email or password.') → redirect to /
```

Security note: Passwords are never stored as plain text. `bcrypt` applies a one-way hash with a random salt, so even if the database file is accessed directly, original passwords cannot be recovered.

6.4 Searching for Parking

```
GET /search?q=302001&type;=covered

→ @login_required ← redirect to login if no valid session cookie

→ Build SQL query: ParkingLot WHERE status='active'

→ If q provided: filter name / address / pin_code (case-insensitive LIKE)

→ If type provided: filter lot_type = 'open' / 'covered' / 'multi-level'

→ For each lot: count free spots (Spot WHERE is_occupied=False)

→ render_template('search-parking.html', lots=lots)

→ Jinja2 loop renders one card per lot with live availability data
```

6.5 Spot Selection Flow


```
GET /spots?lot_id=3
```

- Fetch ParkingLot record by lot_id
- Fetch all Spot records for that lot, ordered by spot_number
- Pass to template: lot, spots list, free_count

In the Browser (JavaScript):

- Jinja2 injects occupied spot IDs into JS: `const occupiedSpots = new Set([...])`
- `buildGrid()` groups spots by row letter (A, B, C, D...)
- Each spot rendered as a div:

Green border = available → onclick calls `selectSpot()`

Red border = occupied → no click handler

- When user clicks a spot:

Selected spot highlighted in blue

'Reserve Spot' panel slides into view

Current IST time auto-filled in the time field

- User clicks 'Confirm Reservation':

Confirmation modal shown with spot + rate details

User confirms → hidden `<form>` submitted via POST `/reserve`

6.6 Reservation (Booking) Flow

```
POST /reserve
```

- Receive: lot_id, spot_number from form
- Re-validate in DB: `Spot WHERE lot_id=X AND spot_number=Y AND is_occupied=False`
(protects against race condition – two users clicking same spot simultaneously)
- If spot gone: flash message → redirect back to spot selection
- If available:

`check_in = datetime.now()` ← server time, NOT client time

```
New Reservation(status='active') added to DB

spot.is_occupied = True

current_user.status = 'parked'

db.session.commit()

→ redirect to /dashboard
```

Why server time? Using `datetime.now()` on the server prevents manipulation — a user cannot change their phone/computer clock to get a cheaper bill.

6.7 Check-Out and Billing Calculation

```
POST /release (user clicks 'Release Spot')

→ Verify: res.user_id == current_user.id ← security ownership check

→ Call _release_reservation(res):

def _release_reservation(res):

    res.check_out = datetime.now()

    secs = (res.check_out - res.check_in).total_seconds()

    res.cost = round((secs / 3600) * res.lot.rate_per_hour, 2)

    res.status = 'completed'

    spot.is_occupied = False

    if no other active reservations: user.status = 'active'

    db.session.commit()

→ redirect to /dashboard
```

Billing formula: Cost (■) = (Seconds parked ÷ 3600) × Rate per hour. This charges per-second (effectively per-minute). Example: 90 minutes at ■40/hr = $90/60 \times 40 = \text{■}60.00$ exactly.

6.8 Admin — Force Release

When an admin clicks **Force Release** on the dashboard live reservations table:

```
POST /admin/release/<res_id>

→ @login_required + @admin_required ← two decorator checks
```

- Same `_release_reservation(res)` helper called
- redirect to `/admin`

The shared `_release_reservation()` helper ensures billing logic is identical whether a user checks out themselves or an admin forces a release.

6.9 Admin — Lot Management (Add / Edit / Delete)

POST `/admin/lots/add`

- Create `ParkingLot` object from form data
- `db.session.flush()` ← assigns new `lot.id` BEFORE commit
- `_make_spots(lot.id, total_spots)` ← auto-generates all spot records
- `db.session.commit()`

POST `/admin/lots/edit/<lot_id>`

- Fetch lot, update fields, commit

POST `/admin/lots/delete/<lot_id>`

- Delete all Spots for this lot (manual cascade)
- Delete all Reservations for this lot
- Delete the lot itself
- `db.session.commit()`

6.10 Reports & Analytics

GET `/admin/reports`

- Query 1: `SUM(cost) WHERE status='completed'` → `total_revenue`
- Query 2: `COUNT(*)` all reservations → `total_res_count`
- Query 3: JOIN User + Reservation
 - `GROUP BY user_id`
 - `ORDER BY SUM(cost) DESC LIMIT 5` → top 5 spenders
- Query 4: `GROUP BY DATE(check_in)`

for last 7 days

→ daily revenue table

→ All are single SQL queries via SQLAlchemy `func.sum()`, `func.count()`

6.11 CSV Export Flow

GET /history/csv (user) OR GET /admin/reports/csv (admin)

→ Fetch reservations from DB

→ Create in-memory `io.StringIO()` buffer (no temp file on disk)

→ Write UTF-8 BOM (`\ufeff`) – makes Excel open ■ symbol correctly

→ Write header row, then one data row per reservation

→ Return Response(

`mimetype='text/csv',`

`Content-Disposition: attachment; filename=parkvault-history.csv`

)

→ Browser automatically triggers file download

7. Authentication & Security Layers

There are **three levels of access control** in ParkVault:

Level	Mechanism	Applied To	Action if Failed
1 — Session check	@login_required decorator	ALL user + admin routes	Redirect to login page (/)
2 — Admin check	@admin_required decorator	All /admin/* routes	Redirect to login page
3 — Ownership check	Logic in route body	Release, Edit, Delete	Silent redirect / flash error

Additional security measures:

- Passwords hashed with bcrypt (salted one-way hash) — never stored as plain text
- Server-side check_in timestamps — billing cannot be manipulated via client clock
- Double-check spot availability at /reserve — prevents race condition double-booking
- Admin cannot delete their own account or demote their own role
- Flask SECRET_KEY signs session cookies — prevents forged sessions

8. Theme System — Dark / Light Mode

The entire UI theming is controlled by a single HTML attribute on the root element:

```
<html data-theme="dark">  ← switches to data-theme="light" on toggle
```

All colours are defined as **CSS Custom Properties (variables)**:

```
:root {

  --glass-bg:    rgba(255,255,255,0.06);

  --text-primary: #e8f0ff;

}

[data-theme='light'] {

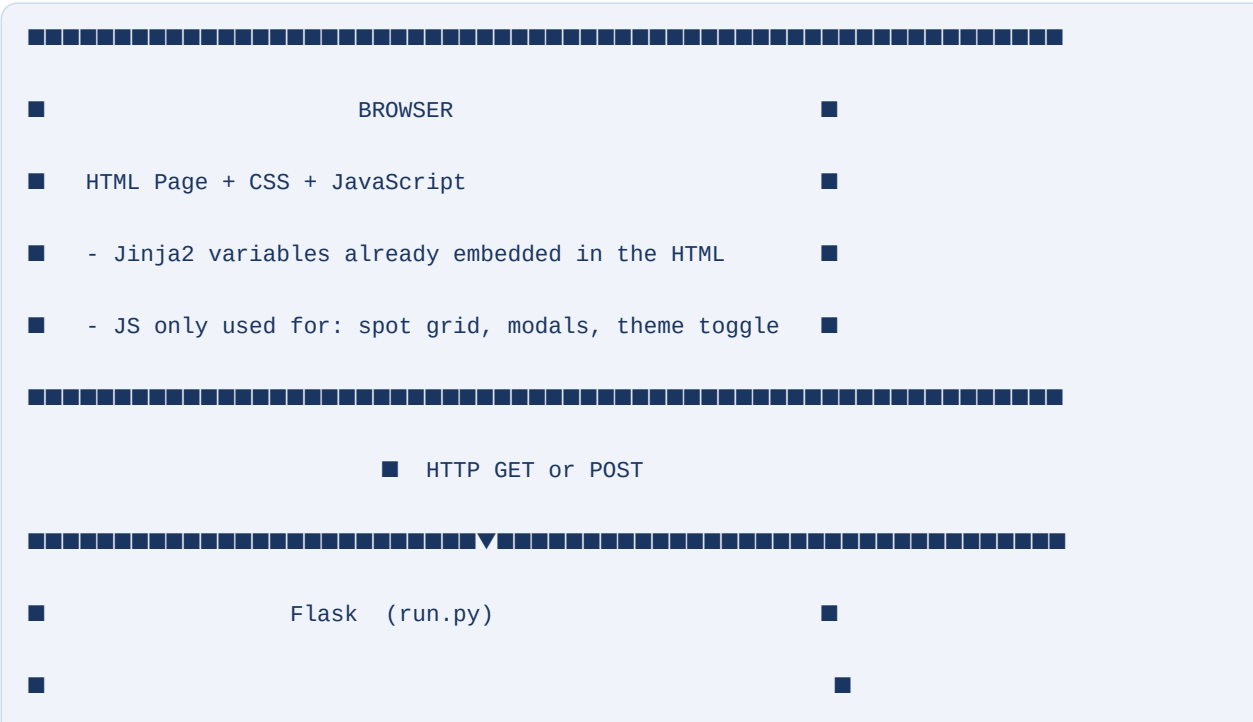
  --glass-bg:    rgba(255,255,255,0.72);

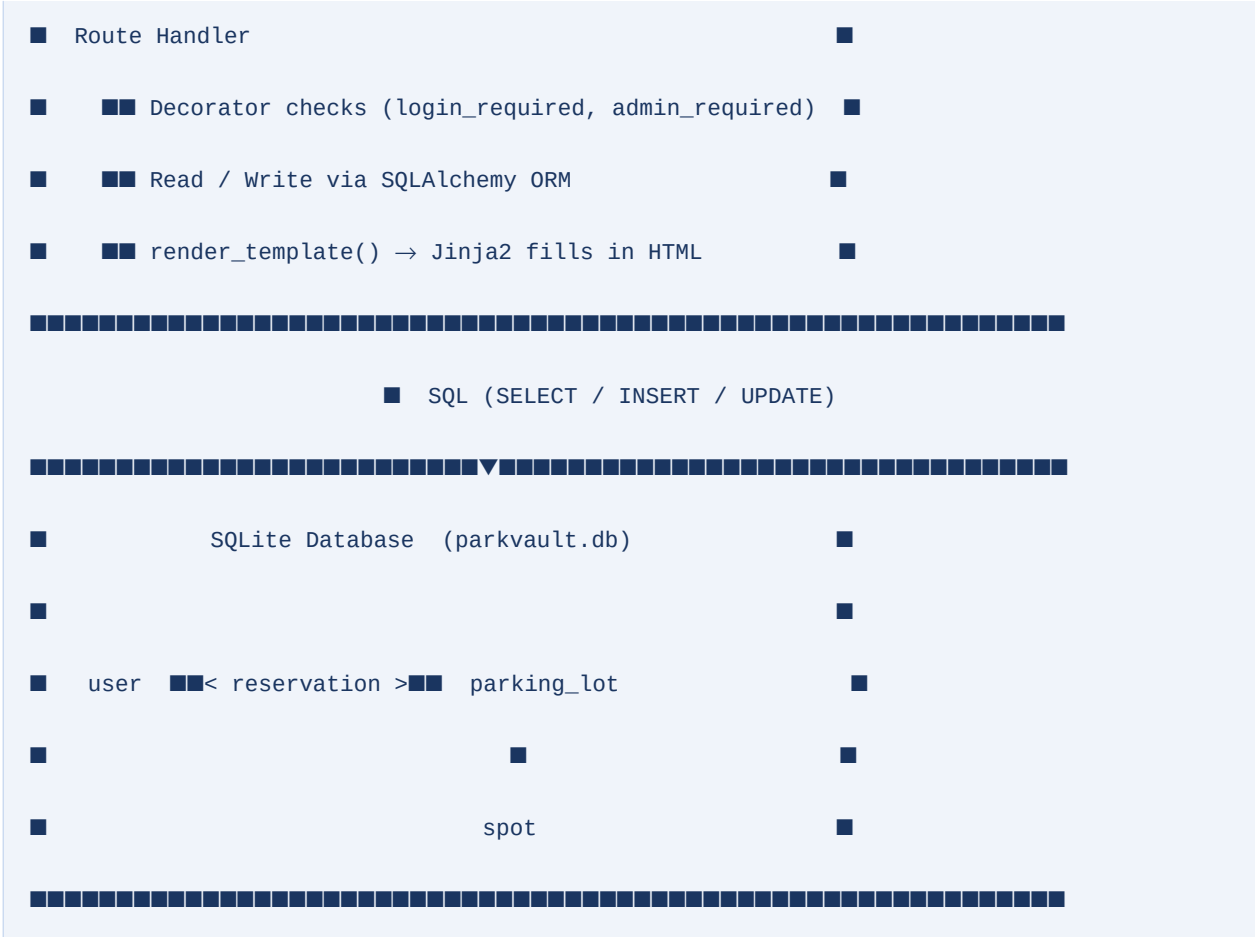
  --text-primary: #0d1527;

}
```

When the user clicks the theme toggle button, JavaScript updates the attribute and saves the preference to **localStorage** — so it persists across page loads and browser sessions. No server round-trip is needed; it is entirely client-side.

9. Data Flow Diagram





10. Key Design Decisions

Design Decision	Reason / Justification
SQLite instead of MySQL / PostgreSQL	Zero-configuration, single file database. Ideal for academic/prototype project — no server setup needed.
Server-side rendering with Jinja2	Simpler than React/Vue/Angular. No separate frontend build pipeline. Entire app runs from one Python file.
bcrypt for password hashing	Industry-standard algorithm. Automatically applies salting, preventing rainbow table and brute-force attacks.
Server datetime.now() for billing	Prevents client-side clock manipulation. A user cannot set their phone to an earlier time to get a cheaper bill.
Shared _release_reservation() helper	DRY (Don't Repeat Yourself) principle — single source of truth for billing logic used by both user checkout and admin force-release.
db.session.flush() before _make_spots()	Assigns the new lot.id within the transaction before commit. Required because Spot records need the FK lot_id immediately.
UTF-8 BOM in CSV export	Ensures Microsoft Excel opens Indian Rupee symbol (₹) and special characters correctly on Windows machines.

Design Decision	Reason / Justification
CSS :has() for overflow scoping	Only table-containing glass panels get horizontal scroll. Form panels and stat card panels are unaffected — no unnecessary scrollbars.
@admin_required as a separate decorator	Keeps route functions clean. Any route can be protected by stacking @login_required + @admin_required without repeating the check logic.
Per-second billing (not per-hour rounding)	Fairer to users. Cost calculated as $(total_seconds / 3600) \times rate$, so a 47-minute session is not rounded up to a full hour.